

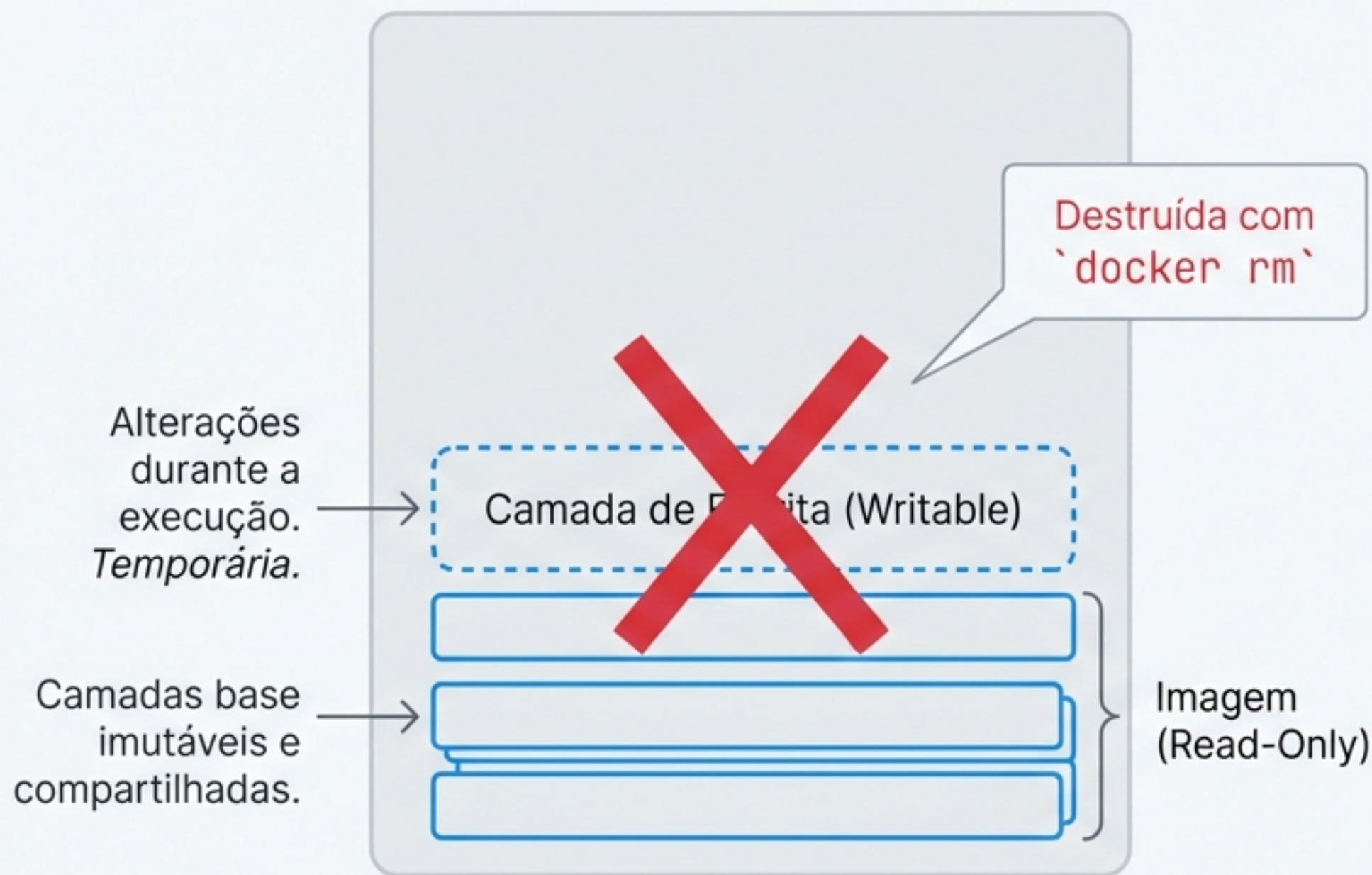
Volumes e Persistência de Dados em Docker

Dominando o armazenamento para aplicações stateful.

Neste guia, vamos resolver o desafio crítico da perda de dados em contêineres. Você aprenderá as ferramentas essenciais para garantir que seus dados sobrevivam ao ciclo de vida efêmero dos contêineres, da prototipagem à produção.

O Problema Central: A Natureza Efêmera dos Contêineres

Arquitetura Copy-on-Write (COW)



Contêineres Docker utilizam um sistema de arquivos em camadas. A imagem base é somente leitura.

Todas as modificações—criação de arquivos, logs, dados de aplicações—acontecem em uma “camada de escrita” superior.

****O Grande Risco****

O Grande Risco*: Esta camada é completamente destruída quando o contêiner é removido, levando todos os seus dados com ela.

Por Que Isso é Um Problema Crítico?

O Banco de Dados em um Contêiner

Imagine um banco de dados PostgreSQL ou MySQL rodando em um contêiner. Cada transação, cada novo registro de usuário, cada log de aplicação é gravado na camada de escrita efêmera.

O Desastre** Se o contêiner for removido—seja por uma atualização, uma falha ou manutenção de rotina—**todos esses dados desaparecem permanentemente.**

A natureza efêmera que torna os contêineres ideais para escalabilidade cria um dilema fundamental para aplicações que precisam guardar estado (stateful).



A Solução: Volumes Docker, Guardiões da Persistência



Volumes são diretórios ou arquivos gerenciados diretamente pelo Docker Engine, projetados especificamente para persistir dados fora do ciclo de vida de um contêiner.

1. Ciclo de Vida Independente

Volumes existem fora do contêiner. Eles persistem mesmo após a execução de ``docker rm``.

2. Gerenciamento pelo Docker

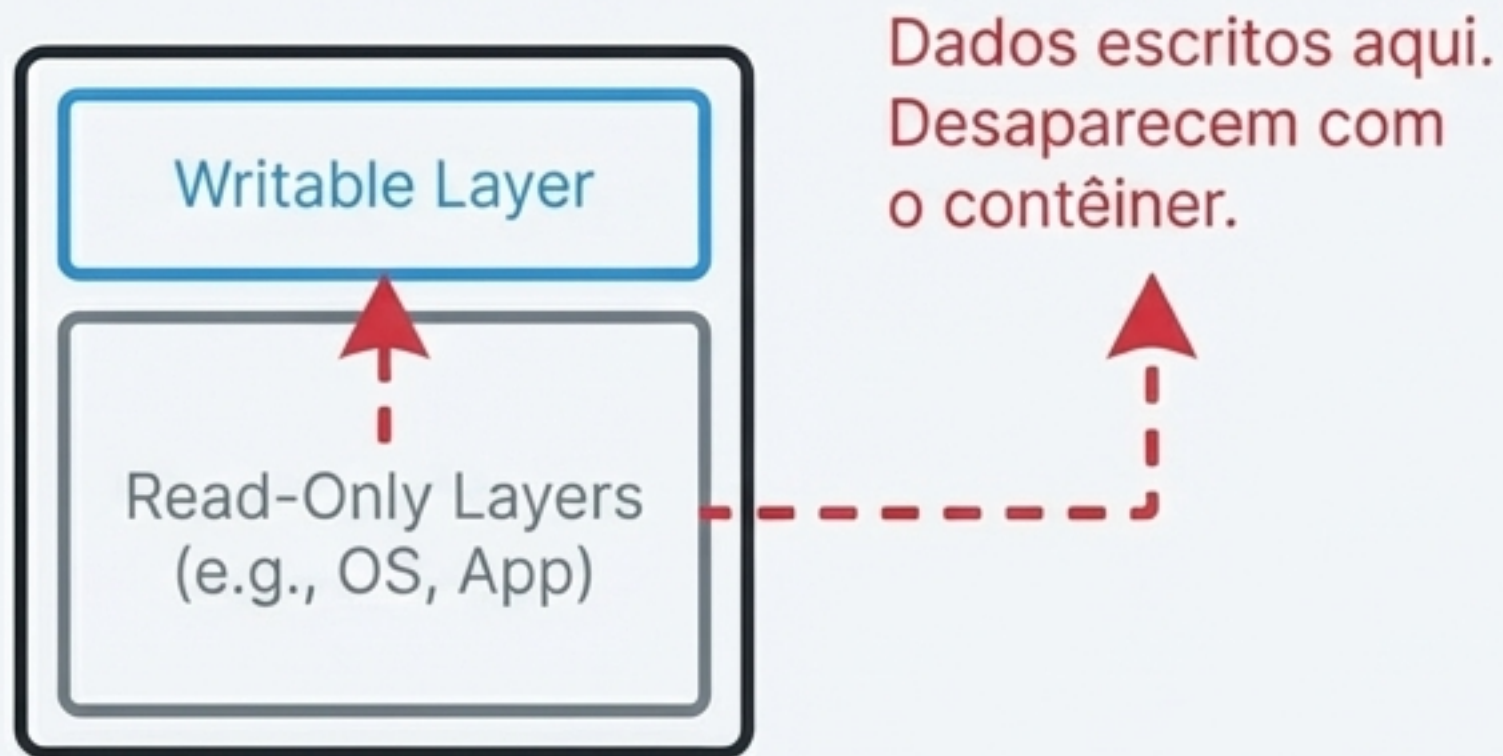
O Docker Engine controla a criação, o armazenamento e a remoção dos volumes.

3. Alto Desempenho

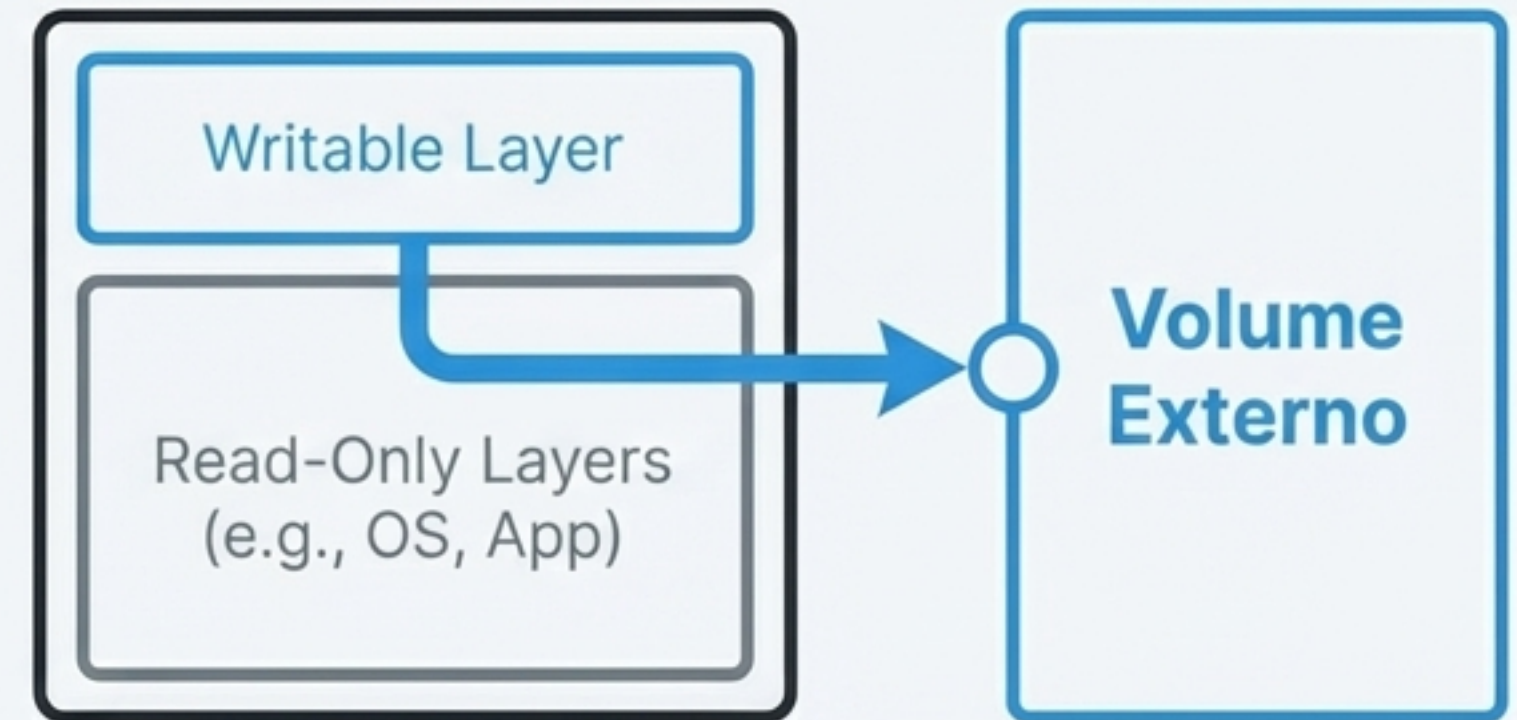
Volumes oferecem performance nativa de disco.

Arquitetura da Solução: Montagem Externa

Sem Volume



Com Volume



Dados são escritos diretamente no volume, que é gerenciado pelo host e sobrevive ao contêiner.

Ferramenta #1: Bind Mounts para Desenvolvimento Ágil



Caso de Uso Principal

Perfeito para o ciclo de desenvolvimento local.

Conceito

Bind Mounts criam um espelhamento direto entre um diretório no seu sistema (host) e um diretório dentro do contêiner.

A Vantagem Mágica

Qualquer alteração feita nos arquivos do seu projeto no host (ex: no VS Code) é **instantaneamente refletida** dentro do contêiner, sem a necessidade de reconstruir a imagem.

Sintaxe do Comando

```
docker run -v [CAMINHO_ABSOLUTO_NO_HOST]:[CAMINHO_NO_CONTÊINER] [IMAGEM]
```

Exemplo Prático

```
# Sincroniza o diretório do projeto local com /app dentro do contêiner \ndocker run -v /home/user/meu-projeto-node:/app node:18
```


Ferramenta #2: Volumes Nomeados para Produção



Caso de Uso Principal

A escolha padrão para produção, especialmente para bancos de dados e dados críticos de aplicações.

Conceito

Com Volumes Nomeados, você apenas fornece um nome (ex: `dados\_db`). O Docker Engine assume controle total de onde e como os dados são armazenados no host.

Benefícios Chave



Abstração: Você não precisa saber o caminho físico no host. Isso torna a sua configuração portátil entre diferentes ambientes.



Segurança: Os dados ficam isolados em uma área de armazenamento gerenciada pelo Docker, protegidos de interferências externas.



Gerenciamento Centralizado: Podem ser criados, listados e removidos de forma independente usando a CLI do Docker.

Dominando a CLI: Comandos Essenciais de Gerenciamento



Criar um Volume

```
docker volume create [NOME_DO_VOLUME]
```

Cria um novo volume nomeado, pronto para ser anexado a um contêiner.

```
docker volume create postgres_data
```

Listar Volumes

```
docker volume ls
```

Exibe todos os volumes gerenciados pelo Docker em seu sistema.

Inspecionar um Volume

```
docker volume inspect [NOME_DO_VOLUME]
```

Fornece metadados detalhados sobre um volume, incluindo seu `Mountpoint`—o caminho real no sistema de arquivos do host.

Olhando “Sob o Capô” com `docker volume inspect`

```
$ docker volume inspect dados_db
```

```
[
  {
    "CreatedAt": "2023-10-27T10:00:00Z",
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/dados_db/_data",
    "Name": "dados_db",
    "Options": {},
    "Scope": "local"
  }
]
```

Name: O nome amigável que você utiliza (`dados\_db`).



Mountpoint: **A informação mais importante.** Este é o caminho exato no sistema de arquivos do host onde o Docker está armazenando fisicamente os dados deste volume.

A Prova Definitiva de Persistência

Vamos demonstrar a independência do ciclo de vida do volume em três passos.



1. Passo 1: Criar e Popular

Lance um contêiner de banco de dados com um volume nomeado. Insira alguns dados de teste.

```
docker run --name db  
-e POSTGRES_PASSWORD=mysecretpassword  
-v pg_data:/var/lib/postgresql/data -d postgres
```



2. Passo 2: Destruir o Contêiner

Remova completamente o contêiner. A camada de escrita efêmera é destruída.

```
docker rm -f db
```



3. Passo 3: Recriar e Verificar

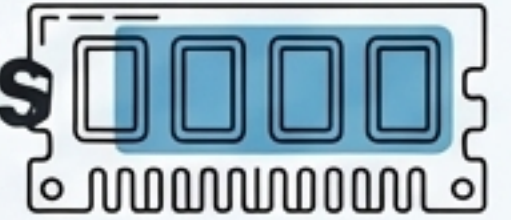
Lance um novo contêiner, anexando o **mesmo volume** pg_data. Conecte-se e verifique os dados.

```
docker run --name db2  
-e POSTGRES_PASSWORD=mysecretpassword  
-v pg_data:/var/lib/postgresql/data -d postgres
```



Os dados originais estarão intactos. A persistência foi comprovada.

Caso de Uso Especial: `tmpfs` para Dados Voláteis



Conceito:

`tmpfs` mounts criam um sistema de arquivos temporário que existe **apenas na memória RAM do host**, nunca no disco.

Características Principais:

- **Velocidade Extrema:** Acesso aos dados na velocidade da RAM.
- **Totalmente Volátil:** Os dados são perdidos assim que o contêiner é parado.
- **Segurança:** Ideal para armazenar segredos, tokens ou dados de cache que não devem, em nenhuma hipótese, ser escritos em disco.

Sintaxe:

```
docker run --tmpfs /caminho/no/contêiner  
[IMAGEM]
```

Exemplo:

```
docker run -d --tmpfs /app/cache nginx
```


Manutenção: Liberando Espaço com `docker volume prune`



O Problema:

Com o tempo, o comando, volumes que não estão mais associados a nenhum contêiner ("volumes órfãos") podem se acumular e consumir espaço em disco.

A Solução:

O comando `prune` remove todos os volumes locais que não estão sendo usados por pelo menos um contêiner.

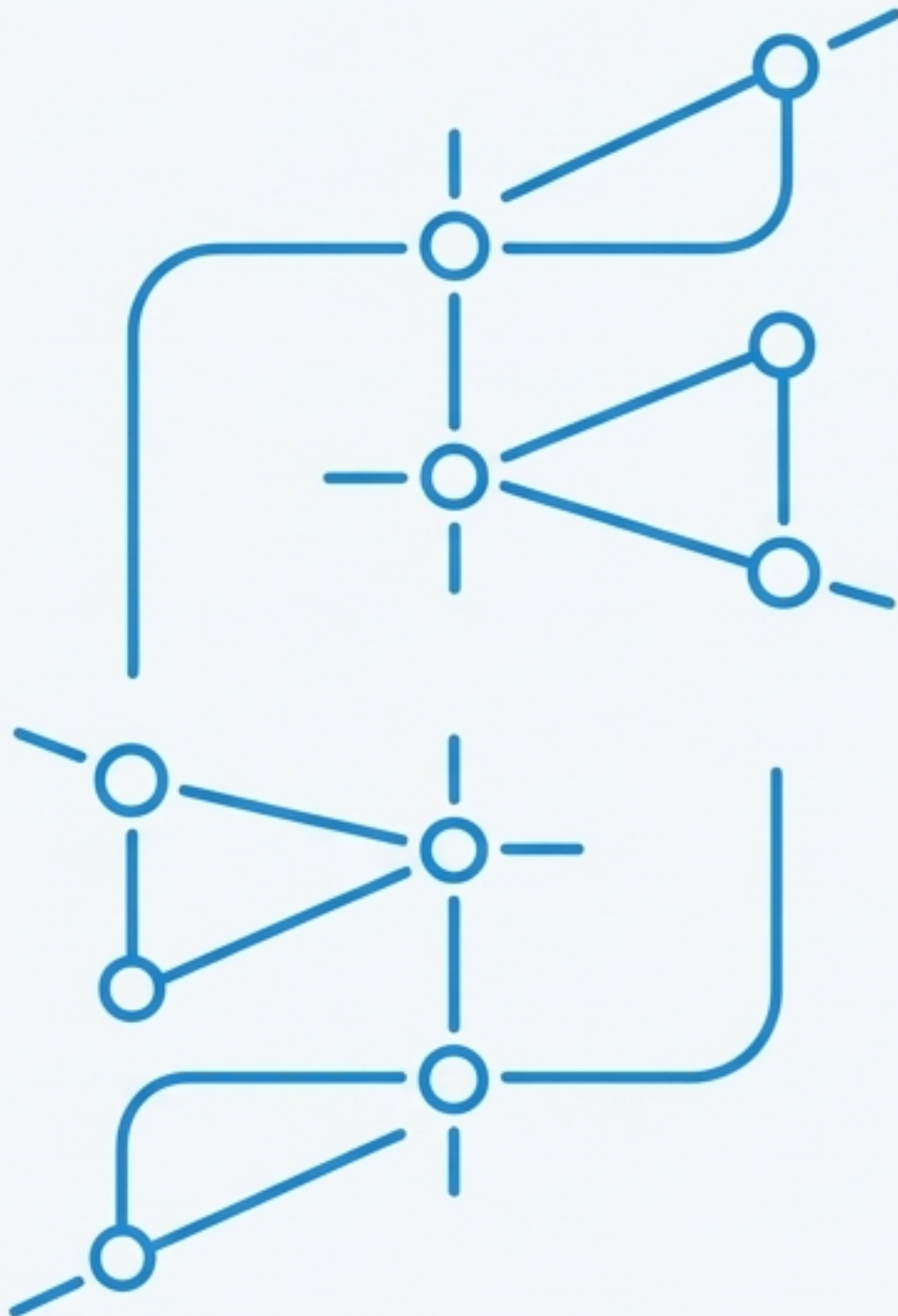
```
docker volume prune
```



ATENÇÃO: Use este comando com extrema cautela. A remoção de um volume é permanente e os dados não podem ser recuperados. Sempre verifique os volumes existentes com `docker volume ls` antes de executar uma limpeza.

Resumo: Qual Tipo de Volume Usar?

Característica	Bind Mount	Volume Nomeado	`tmpfs` Mount
Caso de Uso Ideal	Desenvolvimento Local	Produção / Bancos de Dados	Dados sensíveis / Cache
Localização no Host	Caminho específico definido pelo usuário	Gerenciado pelo Docker (/var/lib/docker/volumes)	Apenas na RAM
Ciclo de Vida	Independente do contêiner	Independente do contêiner	Ligado ao ciclo de vida do contêiner
Portabilidade	Baixa (depende do caminho do host)	Alta (abstraído pelo nome)	Não aplicável
Performance	Alta (nativa de disco)	Alta (nativa de disco)	Extremamente alta (RAM)



Próximos Passos: Da Persistência à Conectividade

Agora que você domina a persistência de dados, o próximo desafio é permitir que seus contêineres se comuniquem de forma segura e eficiente.

- 1. Redes Docker:** Aprenda como contêineres se descobrem e se comunicam em redes isoladas.
- 2. DNS Interno:** Entenda como o Docker fornece resolução de nomes automática entre serviços.
- 3. Aplicação Prática:** Use os conceitos de volumes e redes para construir aplicações multi-contêiner complexas com Docker Compose.